



UNIVERSIDAD DE MONTERREY

Práctica 6

Microprocesadores

M.C. Alejandro Omar Reyes Guía

Nombre: Eugenio Romo GarcíaMat: 612348

Nombre: Ricardo Garza BenitezMat: 645972

“Damos nuestra palabra de que hemos hecho esta actividad con integridad académica”

Monterrey, N. L., a 10 de Junio del 2026

Introducción

En los sistemas embebidos, la comunicación entre el microcontrolador y el usuario suele depender de periféricos de salida visual, siendo las pantallas de cristal líquido (LCD) uno de los dispositivos más utilizados gracias a su bajo costo, facilidad de control y versatilidad para mostrar texto, números y símbolos definidos por el propio programador. Estas pantallas se conectan típicamente mediante un bus de datos paralelo y un conjunto reducido de líneas de control, lo que permite integrarlas con microcontroladores de gama media, como el PIC16F887, sin necesidad de hardware adicional complejo.

En esta práctica se trabajó con el microcontrolador PIC16F887 y una pantalla LCD LM016L de 16x2 caracteres, programando ambos en lenguaje C mediante el compilador XC8 y apoyándose en una librería de funciones para el manejo del LCD. La práctica se dividió en dos actividades: en la primera se inicializó la pantalla en modo de 4 bits para desplegar el mensaje "HELLO WORLD!" y generar de manera secuencial los caracteres ASCII de la letra A a la P; en la segunda actividad se crearon dos caracteres personalizados mediante la memoria CGRAM del controlador del LCD (un corazón y una carita) y se programó un sistema que alterna entre dos mensajes propios mediante la pulsación de un botón con antirrebote por software.

El objetivo general de esta práctica es familiarizarse con el protocolo de comunicación entre el microcontrolador y el LCD, comprender la relación entre la frecuencia del oscilador externo y los tiempos de retardo definidos en el código, y aplicar el manejo de entradas digitales para controlar de forma interactiva el contenido desplegado en pantalla.

Marco teórico

Pantalla de cristal líquido LM016L y el controlador HD44780

La pantalla LM016L es un módulo LCD alfanumérico de 16 columnas por 2 filas, basado en el controlador HD44780, uno de los controladores de pantallas de cristal líquido más utilizados en aplicaciones con microcontroladores. Este controlador integra internamente un generador de caracteres en ROM (CGROM) que contiene los patrones de mapas de bits de letras, números y símbolos estándar, así como una memoria de datos de despliegue (DDRAM) que almacena el carácter que debe mostrarse en cada posición de la pantalla [2].

El módulo cuenta con 14 terminales: dos de alimentación (VSS y VDD), una terminal de contraste (VEE) que normalmente se conecta a un potenciómetro, tres líneas de control (RS, RW y E) y ocho líneas de datos (D0 a D7). La línea RS determina si la información enviada corresponde a un comando o a un dato a desplegar, la línea RW selecciona entre lectura y escritura, y la línea E habilita la transferencia de información mediante un pulso. En el circuito de esta práctica, las líneas RS y E del LCD se conectaron a los pines RC2 y RC3 del PIC16F887, mientras que las líneas de datos utilizadas (D4 a D7) se conectaron a RC4-RC7 [1].

Comunicación en modo de 4 bits

Para reducir el número de líneas de conexión entre el microcontrolador y el LCD, el controlador HD44780 permite operar en modo de 4 bits, en el cual cada byte de comando o dato se transmite en dos pasos sucesivos: primero el nibble más significativo y después el menos significativo, sincronizados mediante pulsos en la línea E. Esta configuración reduce de ocho a cuatro las líneas de datos necesarias, a costa de duplicar el número de transferencias por byte [2].

En el código de ambas actividades, la inicialización de la pantalla se realiza mediante la estructura `LCD lcd = {&PORTC, 2, 3, 4, 5, 6, 7}`, en la cual se asocia el puerto C del microcontrolador con las terminales RS, EN, D4, D5, D6 y D7 del LCD. La función `LCD_Init` configura internamente la secuencia de inicialización recomendada por el fabricante, estableciendo el modo de 4 bits, el número de líneas de la pantalla y el formato de los caracteres, de manera que el resto del programa solo necesita invocar funciones como `LCD_Clear`, `LCD_Set_Cursor` y `LCD_puts` [1].

Oscilador de cristal y temporización con `_XTAL_FREQ`

El PIC16F887 puede operar con distintos tipos de osciladores, entre ellos el modo HS (High Speed), utilizado cuando se conecta un cristal de cuarzo externo de alta frecuencia, en este caso de 16 MHz, junto con sus capacitores de carga correspondientes en las terminales OSC1 y OSC2. La directiva `#pragma config FOSC = HS` indica al compilador que el reloj del sistema proviene de este cristal externo [1].

La librería del compilador XC8 utiliza la macro `_XTAL_FREQ` para calcular el número de ciclos de instrucción necesarios en las funciones de retardo `__delay_ms` y `__delay_us`. Si el valor declarado en `_XTAL_FREQ` no corresponde a la frecuencia real del cristal instalado en el circuito, todos los retardos generados por el programa se ven afectados de manera proporcional, lo cual resulta crítico en la comunicación con el LCD, ya que esta depende de tiempos mínimos de configuración y escritura especificados por el fabricante del controlador [2].

Memoria CGRAM y caracteres personalizados (Actividad 2)

Además de los caracteres predefinidos almacenados en la memoria CGROM, el controlador HD44780 incluye una memoria CGRAM (Character Generator RAM) que permite al usuario definir hasta ocho caracteres personalizados de 5x8 píxeles. Cada carácter se define mediante ocho bytes, donde los cinco bits menos significativos de cada byte representan el estado, encendido o apagado, de los píxeles de una fila del carácter [2].

Para escribir en la CGRAM, primero se envía un comando que establece la dirección de memoria deseada mediante la instrucción `0x40 | (posición << 3)`, y posteriormente se transfieren los ocho bytes del patrón de bits como si fueran datos a desplegar. En la segunda actividad, esta técnica se utilizó para definir dos caracteres personalizados, un corazón y una carita, los cuales posteriormente se invocan dentro del texto desplegado

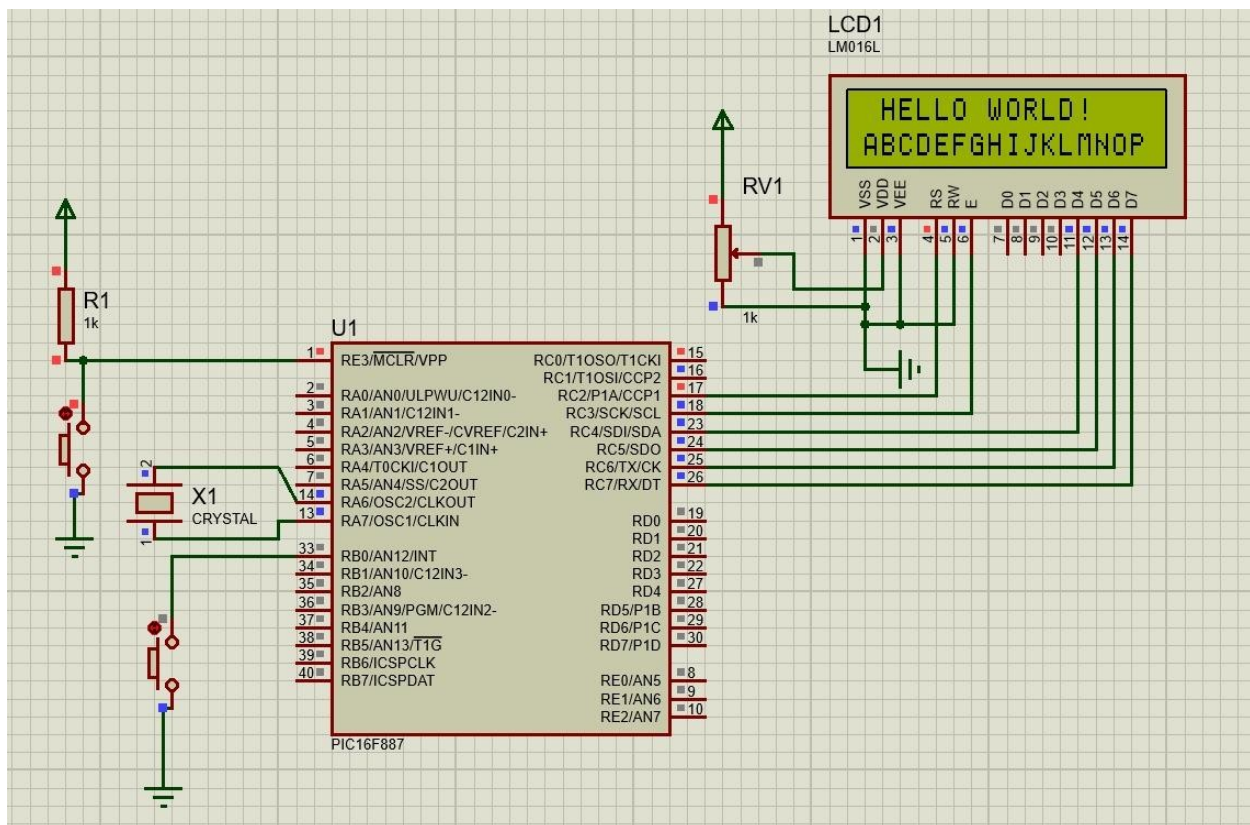
mediante su código de posición (0 y 1) en la CGRAM, a través de la función LCD_putc [1].

Entradas digitales, resistencias de pull-up y antirrebote por software (Actividad 2)

El PIC16F887 cuenta con módulos de conversión analógico-digital que, por configuración predeterminada, asignan ciertos pines del PORTB como entradas analógicas. Para que estos pines puedan utilizarse como entradas o salidas digitales, es necesario configurar los registros ANSEL y ANSELH, los cuales determinan si cada pin opera en modo analógico o digital [1].

En la segunda actividad, el pin RB0, compartido con la entrada analógica AN12, se configuró como entrada digital mediante `ANSELH = 0` y `TRISBbits.TRISB0 = 1`, habilitando además la resistencia de pull-up interna del puerto B mediante `OPTION_REGbits.nRBPU = 0` y `WPUBbits.WPUB0 = 1`. Para evitar que los rebotes mecánicos del botón generen múltiples lecturas falsas durante una sola pulsación, se implementó un antirrebote por software mediante un retardo de 20 milisegundos antes y después de detectar el cambio de estado del botón, técnica que ya se había aplicado en prácticas anteriores con resultados satisfactorios [2].

Simulación - Actividad 1



Simulación - Actividad 2

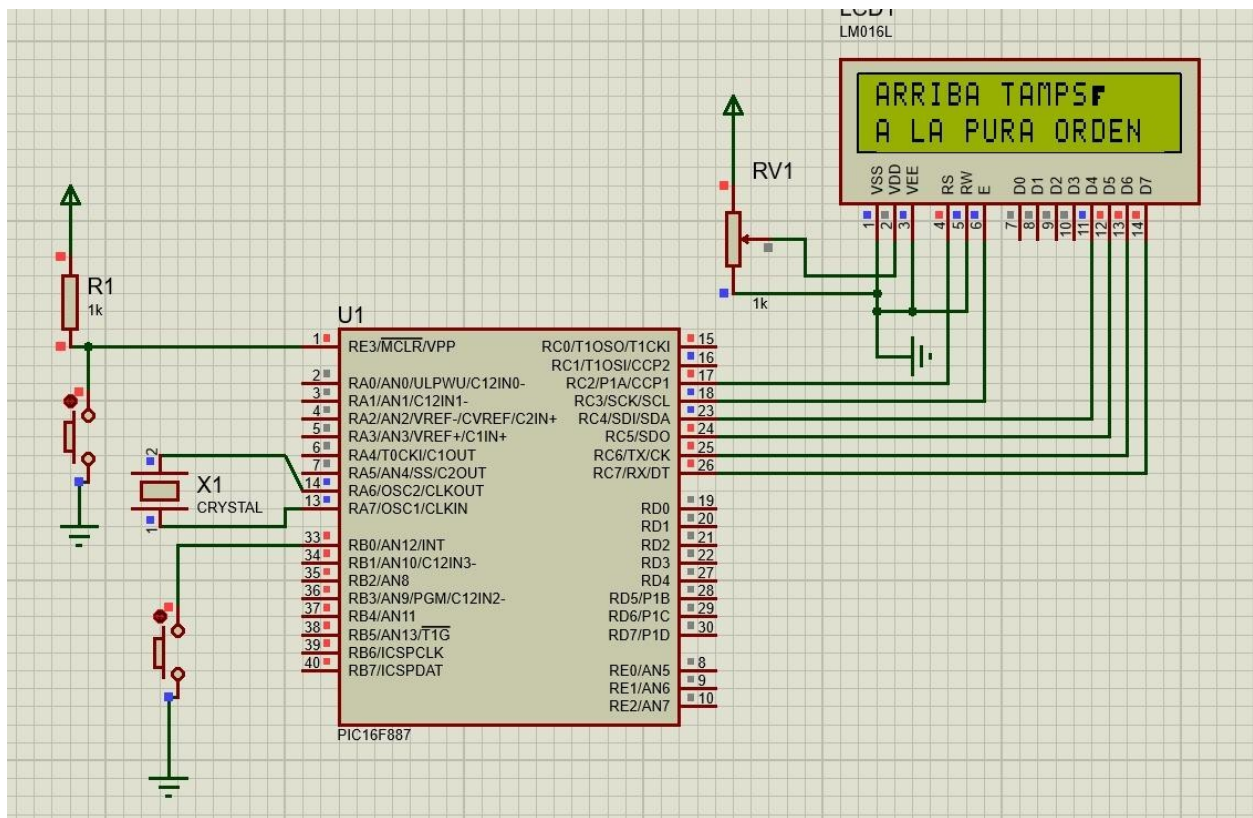
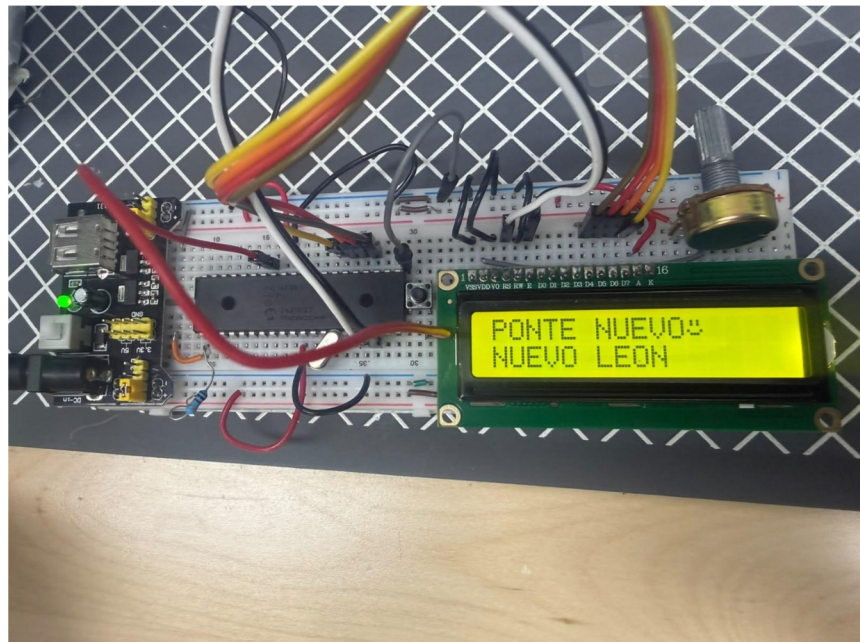


Foto del circuito en Físico



Resultados

Actividad 1: Inicialización del LCD y generación de caracteres

Al energizar el circuito, la pantalla LCD muestra en la primera línea el mensaje "HELLO WORLD!" y, en la segunda línea, comienza a desplegar de manera secuencial los caracteres ASCII desde la letra A hasta la letra P, con un retardo aproximado de 300 milisegundos entre cada uno. Una vez completada la secuencia, el programa espera un segundo adicional, limpia la pantalla y reinicia el ciclo, comportamiento que se mantuvo constante durante las pruebas tanto en la simulación de Proteus como en el circuito armado físicamente.

Durante las primeras pruebas en físico, la pantalla mostraba caracteres ilegibles y deformados, sin relación aparente con el texto programado. Al revisar el circuito y el código se identificó que el archivo de cabecera de la librería del LCD aún tenía configurado un cristal de 4 MHz, mientras que el cristal instalado físicamente era de 16 MHz. Esta diferencia provocaba que los retardos calculados con `_XTAL_FREQ` fueran incorrectos, desfasando los pulsos enviados a la línea E y generando escrituras incompletas o mal sincronizadas en el LCD.

Una vez corregido el valor del cristal a 16 MHz tanto en la directiva `_XTAL_FREQ` como en la configuración interna de la librería, el LCD comenzó a mostrar el texto correctamente, validando que el cableado de las líneas RS, E y D4-D7 hacia el puerto C era el adecuado y que el modo de 4 bits quedó configurado de forma correcta.

Actividad 2: Caracteres personalizados y mensajes alternables

Al iniciar el programa, la pantalla muestra en la primera línea el texto "PONTE NUEVO" seguido del carácter personalizado del corazón, y en la segunda línea el texto "PONTE LEON", formando el mensaje completo "PONTE NUEVO LEON". Al presionar el botón conectado a RB0, la pantalla cambia al segundo mensaje, mostrando "ARRIBA TAMPAS" junto con el carácter de la carita en la primera línea, y "A LA PURA ORDEN" en la segunda línea. Cada nueva pulsación del botón alterna entre ambos mensajes.

Los dos caracteres personalizados, definidos mediante sus mapas de bits de 5x8 y cargados en las posiciones 0 y 1 de la CGRAM, se desplegaron correctamente con la forma de un corazón y de una carita sonriente, lo que confirmó que la rutina `LCD_CustomChar` y la dirección de memoria calculada con `0x40 | (pos << 3)` funcionaron de acuerdo con lo descrito en el marco teórico.

En las primeras pruebas con el botón, antes de implementar el antirrebote, se observaron cambios de mensaje inconsistentes: en ocasiones una sola pulsación alternaba el mensaje varias veces seguidas debido al rebote mecánico del contacto. Al agregar los retardos de 20 milisegundos antes y después de detectar el cambio de estado en RB0, el cambio de mensaje se volvió consistente, alternando una sola vez por cada pulsación del botón.

Conclusiones Individuales

Ricardo Garza:

Para mí, el momento más formativo de esta práctica fue el problema que tuvimos con el cristal de 16 MHz. En la simulación todo funcionaba sin problema, pero al armar el circuito en físico la pantalla mostraba puros caracteres deformados, y al inicio pensamos que era un error de cableado o de la librería. Revisando con calma nos dimos cuenta de que el archivo de cabecera del LCD seguía configurado para un cristal de 4 MHz, mientras que en la tarjeta físicamente teníamos uno de 16 MHz. Ese desfase entre lo que dice el código y lo que realmente está soldado en la placa cambió por completo los tiempos de los pulsos hacia la línea E del LCD, y para mí esto se conecta directamente con lo que aprendimos en la práctica anterior sobre revisar con cuidado los `#pragma config`: no basta con que el código compile, también tiene que coincidir con el hardware real que tenemos enfrente.

También me quedó muy claro el funcionamiento de la CGRAM al armar los caracteres personalizados. Tener que pensar el corazón y la carita como una matriz de 5x8 y traducir cada fila a un valor hexadecimal me ayudó a entender que, a fin de cuentas, toda la pantalla es solo memoria que el controlador del LCD interpreta como píxeles. Y al conectar el botón con la resistencia de pull-up interna, comprobé en carne propia por qué el antirrebote por software es necesario: sin el retardo, una sola pulsación cambiaba el mensaje varias veces, y con los 20 ms agregados el comportamiento se volvió predecible. En general, esta práctica reforzó la idea de que la teoría y el hardware tienen que verificarse juntos, no por separado.

Eugenio Romo:

En esta práctica pude entender mejor cómo se comunica el microcontrolador con el LCD usando solo cuatro líneas de datos en lugar de ocho, gracias al modo de 4 bits del controlador HD44780. Ver cómo la estructura `Lcd = {&PORTC, 2, 3, 4, 5, 6, 7}` relaciona directamente los pines del PIC con las terminales RS, EN y D4-D7 me ayudó a visualizar de forma más concreta algo que antes solo conocía de manera teórica.

También me pareció interesante cómo funciona la memoria CGRAM para crear caracteres propios, y cómo basta con cargar ocho bytes en la dirección correcta para que el LCD dibuje una figura nueva, como el corazón o la carita que usamos. El detalle del cristal de 16 MHz también me dejó claro que la frecuencia del oscilador no es solo un dato que se pone en el código por costumbre, sino que afecta directamente todos los tiempos de espera del programa, y por lo tanto debe coincidir siempre con el componente físico que realmente está en el circuito.

Referencias:

[1] Microchip Technology. (2003). PIC16F887 datasheet. <https://ww1.microchip.com/downloads/en/DeviceDoc/41291D.pdf>

[2] Hitachi Ltd. (1998). HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver datasheet. <https://www.sparkfun.com/datasheets/LCD/HD44780.pdf>